

```
slinux@freedom:~$ make
gcc -I . -c module.c
gcc main.o module.o -o target_bin
```

Since *module.c* in the dependency tree has changed (it now has a later timestamp than its target, *module.o*), *make* runs the action for the *module.o* target, which recompiles the changed source file. It then runs the action for the *all* target.

We can explicitly invoke the *clean* target to clean up all the generated *.o* files and *target_bin*:

```
$ make clean
rm -rf *.o
rm target_bin
```

More bytes on Makefiles

Make provides many interesting features that we can use in *Makefiles*. Let's look at the most essential ones.

Dealing with assignments

There are different ways of assigning variables in a *Makefile*. They are (type of assignment, followed by the operator in parentheses):

1. Simple assignment (*:=*)

We can assign values (RHS) to variables (LHS) with this operator, for example: *CC := gcc*. With simple assignment (*:=*), the value is expanded and stored to all occurrences in the *Makefile* when its first definition is found.

For example, when a *CC := \$(GCC) \$(FLAGS)* simple definition is first encountered, *CC* is set to *gcc -W* and wherever *\$(CC)* occurs in actions, it is replaced with *gcc -W*.

2. Recursive assignment (*=*)

Recursive assignment (the operator used is *=*) involves variables and values that are not evaluated immediately on encountering their definition, but are re-evaluated every time they are encountered in an action that is being executed. As an example, say we have:

```
GCC = gcc
FLAGS = -W
```

With the above lines, *CC = \$(GCC) {FLAGS}* will be converted to *gcc -W* only when an action like *\$(CC) file.c* is executed somewhere in the *Makefile*. With recursive assignment, if the *GCC* variable is changed later (for example, *GCC = c++*), then when it is next encountered in an action line that is being updated, it will be re-evaluated, and the new value will be used; *\$(CC)* will now expand to *c++ -W*. We will also have an interesting and useful application further in the article, where this feature is used to deal with varying cases of filename extensions of image files.

3. Conditional assignment (*?:*)

Conditional assignment statements assign the given value to the variable only if the variable does not yet have a value.

4. Appending (*+=*)

The appending operation appends texts to an existing variable. For example:

```
CC = gcc
CC += -W
CC now holds the value gcc -W.
```

Though variable assignments can occur in any part of the *Makefile*, on a new line, most variable declarations are found at the beginning of the *Makefile*.

Using patterns and special variables

The *%* character can be used for wildcard pattern-matching, to provide generic targets. For example:

```
%.o: %.c
[Tab] actions
```

When *%* appears in the dependency list, it is replaced with the same string that was used to perform substitution in the target.

Inside actions, we can use special variables for matching filenames. Some of them are:

\$@ (full target name of the current target)

\$? (returns the dependencies that are newer than the current target)

*\$** (returns the text that corresponds to *%* in the target)

\$< (name of the first dependency)

\$^ (name of all the dependencies with space as the delimiter)

Instead of writing each of the file names in the actions and the target, we can use shorthand notations based on the above, to write more generic *Makefiles*.

Action modifiers

We can change the behaviour of the actions we use by prefixing certain *action modifiers* to the actions. Two important action modifiers are:

- (**minus**) Prefixing this to any action causes any error that occurs while executing the action to be ignored. (By default, execution of a *Makefile* stops when any command returns a non-zero (error) value.) If an error occurs, a message is printed, with the status code of the command, and noting that the error has been ignored. Looking at the *Makefile* from our sample project: in the *clean* target, the *rm target_bin* command will produce an error if that file does not exist (this could happen if the project had never been compiled, or if *make clean* is run twice consecutively). To handle this, we can prefix the *rm* command with a minus, to ignore errors: *-rm target_bin*.

@ (**at**) suppresses the standard print-action-to-standard-output behaviour of *make*, for the action/command that is prefixed with *@*. For example, to echo a custom message to standard output, we want only the output of the *echo* command, and don't want to print the *echo* command line itself. *@echo Message* will print "Message" without the *echo* command line being printed.